PA 2

-Group 31

1. Changes in e1000_main.c

   a) e1000_xmit_frame: We first get the iphdr and udphdr using the functions provided by skb. Ensure that the destination of the packet we are intercepting is sent to the dummy IP address 100.100.100.100. Once found, in this packet, we update the iphdr protocol to ICMP, change the destination IP to the first 4 bytes of the payload. Set ip_summed to complete to let the NIC know we have already computed the checksum. Now, typecast the udphdr to an icmphdr, as the size of the headers remains the same. In the newly created icmphdr, we set the fields. Type to echo request, id to the source port that we had saved from udphdr, code to 0. Now both iphdr and icmphdr checksums are set to 0, and the checksum is recomputed using the function provided in class.

   Added code: at the end of the doc

   b) e1000_receive_skb: we reset the skb and get the iphdr. Check if the packet is icmp protocol. Since there is no function for icmphdr we use iphdr address + the length of ipheader as the position of icmphdr. The payload exists contiguous after the icmp, i.e icmphdr + len(icmphdr). Check if the payload contains DECAF, as that lets us know if this is indeed the icmp packet sent by udpping. Now that we have our packet, we cast icmp to a new udphdr. We set the destination port to the source port that we had saved in icmp->id. Len is recalculated as len(udphdr) + size of payload. Iphdr protocol is updated to udp. All that's left is a checksum for which we need to create a pseudo header which we define as a new struct within the code. Set all the fields from the iphdr and create a buffer using kmalloc and append psuedohdr followed by udphdr and payload as checksum is calculated. IP checksum is calculated normally using the checksum function we have been using. Set ip_summed to checksum_none to inform the kernel that we have manually calculated the checksum.

   Added code: at the end of the doc

2) Output After udpping to google

```
shivank@shivank-Standard-PC-i440FX-PIIX-1996:~$ ping www.google.com
PING www.google.com (172.217.26.100) 56(84) bytes of data.
64 bytes from kix05s01-in-f4.1e100.net (172.217.26.100): icmp_seq=1 ttl=255 time
=3.28 ms
^C
--- www.google.com ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 3.278/3.278/3.278/0.000 ms
shivank@shivank-Standard-PC-i440FX-PIIX-1996:~$ ./udpping 172.217.26.100
UDP echo: 172.217.26.100
Congrats: test passed
shivank@shivank-Standard-PC-i440FX-PIIX-1996:~$
```

First, use the ping command to find the IP address of Google. Using this IP address, we ./udpping (Google's IP), as seen in the screenshot, the test passes, implying that Google received the icmp packet and returned an icmp reply with the payload containing our magic number.

3) Output ping to Google after udpping to Google

```
shivank@shivank-Standard-PC-i440FX-PIIX-1996:~$ ping www.google.com
PING www.google.com (172.217.26.100) 56(84) bytes of data.
64 bytes from kix05s01-in-f4.1e100.net (172.217.26.100): icmp_seq=1 ttl=255 time
=3.28 ms
^C
--- www.google.com ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 3.278/3.278/3.278/0.000 ms
shivank@shivank-Standard-PC-i440FX-PIIX-1996:~$ ./udpping 172.217.26.100
UDP echo: 172.217.26.100
Congrats: test passed
shivank@shivank-Standard-PC-i440FX-PIIX-1996:~$ ping www.google.com
PING www.google.com (172.217.26.100) 56(84) bytes of data.
64 bytes from kix05s01-in-f4.1e100.net (172.217.26.100): icmp_seq=1 ttl=255 time
=3.19 ms
64 bytes from kix05s01-in-f4.1e100.net (172.217.26.100): icmp_seq=2 ttl=255 time
=3.75 ms
^C
--- www.google.com ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 3.190/3.469/3.749/0.279 ms
shivank@shivank-Standard-PC-i440FX-PIIX-1996:~$
```

The ping Google command, followed by the udpping command, works as intended. The changes in e1000 were only altered after all the checks were done, ensuring the packet was indeed sent by udpping and to be received by udpping.

4) Name and Roll numbers

Altamash Hasnain	2023073
Dhruv Jain	2023199
Shivank Rajput	2023508
Divyanshi	2023209
Saksham Arora	2023466

E1000_xmit_frame addition

```
struct iphdr *iph = ip_hdr(skb);
    if (iph) {

        // __be32 saddr = iph->saddr;
        __be32 daddr = iph->daddr;
        // printk(KERN_INFO "src IP: %pI4, dst IP: %pI4\n", &saddr,
&daddr);
        struct udphdr *udph = udp_hdr(skb);
        unsigned char *payload = NULL;
        if(udph){
            // printk(KERN_INFO "yooo\n");
            if(daddr==htonl(0x64646464)){
                unsigned *magic;
                skb->ip_summed=CHECKSUM_COMPLETE;
                payload = (unsigned char *)(udph + 1);
                __be32 target_ip = *(__be32 *)payload;
                iph->daddr=target_ip;
                iph->protocol=1;
                iph->check=0;
                __u16 srcport = udph->source;
                // printk("%c \n",payload);

                // printk("%pI4 \n",&target_ip);
                // memcpy(target_ip,payload,4);
                // payload=payload<<32;
                // printk("oy\n");

                struct icmphdr *icmph = (struct icmphdr *)udph;
                // printk("uwuwuwuuw\n");
```

```
                icmph->type=ICMP_ECHO;
                icmph->code=0;
                icmph->un.echo.id=srcport;
                // icmph->un.echo.id=0;
                // printk("%d \n",udph->source);
                icmph->un.echo.sequence=htons(1);
                icmph->checksum=0;
                // magic = (unsigned*)(icmph + 1);


                // *magic= 0xDECAF;
                icmph->checksum = checksum((unsigned short*)icmph, 8);
                // printk("lmfaomlmfaomflamfalm\n");
                // printk("%d \n",icmph->checksum);


                // recompute IP checksum
                iph->check = checksum((unsigned short*)iph, (iph->ihl)*4
/2);
                // printk("hehehehehehehehh\n");
                // printk("ours %d \n",icmph->checksum);

                // printk("%d \n",iph->ihl*4);


            }

        }
    }
```

E1000_receive_skb changes

```
skb->protocol = eth_type_trans(skb, adapter->netdev);

    if (skb->protocol == htons(ETH_P_IP)) {
        skb_reset_network_header(skb);

        struct iphdr *iph = ip_hdr(skb);
```

```
        if (iph) {
            // printk("dst ip = %pI4\n", &iph->daddr);
            // printk("proto  = %u\n", iph->protocol);
            if(iph->protocol==1){
                printk("src ip = %pI4\n", &iph->saddr);
                // struct icmphdr *icmph = (struct icmphdr*) (iph +
iph->ihl*4);
                int iphdr_len = iph->ihl * 4;
                u8 *transport_header = (u8 *)iph + iphdr_len;


                struct icmphdr *icmph = (struct icmphdr
*)transport_header;
                u8 *payload = (u8 *)icmph +sizeof(struct icmphdr);
                __be32 magic = *(__be32 *)(payload + 4);

                // printk("magic raw=0x%08x host=0x%08x\n",
                //          magic, ntohl(magic));
                if (magic == 0x000DECAF) {
                    // printk("im gonna kms\n");

                    __u16 port =icmph->un.echo.id;
                    int iphdr_len= iph->ihl*4;
                    int icmp_payload_len = ntohs(iph->tot_len) -iphdr_len
-sizeof(struct icmphdr);
                    int udp_len= sizeof(struct udphdr)+icmp_payload_len;


                    struct udphdr *udph = (struct udphdr *)icmph;
                    udph->source= port;
                    udph->dest=port;
                    udph->len=htons(udp_len);
                    udph->check=0;


                    iph->protocol=IPPROTO_UDP;


                    struct {
                        __be32 saddr;
```

```
                        __be32 daddr;
                        __u8  zero;
                        __u8 protocol;
                        __be16 udp_len;
                    } __attribute__((packed)) psh;

                    psh.saddr=iph->saddr;
                    psh.daddr=iph->daddr;
                    psh.zero=0;
                    psh.protocol=IPPROTO_UDP;
                    psh.udp_len=htons(udp_len);

                    int buf_len= sizeof(psh) + udp_len;
                    u8 *buf= kmalloc(buf_len, GFP_ATOMIC);
                    if (!buf) {
                        return;
                    }

                    memcpy(buf, &psh, sizeof(psh));
                    memcpy(buf + sizeof(psh), udph, udp_len);


                    udph->check = checksum((unsigned short *)buf,
(buf_len+1)/2);
                    if (udph->check==0)
                        udph->check=0xFFFF;

                    kfree(buf);

                    iph->check=0;
                    iph->check=checksum((unsigned short *)iph,
iph->ihl*2);

                    skb_set_transport_header(skb, iphdr_len);
                    skb->ip_summed=CHECKSUM_NONE;
                }

                // pr_info("first 8 payload bytes: %02x %02x %02x %02x
%02x %02x %02x %02x\n",
                //  payload[0], payload[1], payload[2], payload[3],
```